# M08: Users, Groups & Sudo

> "Your team is growing. Five new developers start Monday. Set up their accounts, assign the right permissions, and give them the power they need — no more, no less."

# Why User Management Matters

**Scenario:** Your team shares a production server.

| Without User Management | With User Management |
|---|---|
| Everyone uses root → no accountability | Each person has their own account |
| Security breach = full system access | Compromised account = limited damage |
| Team member leaves → can't revoke access | One command: disable their account |
| Compliance audit fails | Clear audit trail: who did what |

> **Cloud connection:** AWS IAM, GCP IAM, Azure AD — all follow the same principle: **individual accounts, least privilege, audit trails**. Learn it here, use it everywhere.

# How Linux Identifies Users

Linux doesn't care about names — it cares about **numbers** (UIDs).

| Concept | What It Is | Example |
|---|---|---|
| **UID** | User ID — unique number for each user | `1000` |
| **GID** | Group ID — primary group number | `1000` |
| **Username** | Human-friendly label mapped to UID | `anas` |

**Special UIDs:**

| UID | User | Prompt |
|---|---|---|
| `0` | `root` — superuser, unrestricted | `#` |
| `1-999` | System accounts — services like `www-data`, `sshd` | `$` |
| `1000+` | Regular users — human accounts | `$` |

> This is why your prompt shows `$` (regular user, UID 1000+) or `#` (root, UID 0). The prompt character reflects who you are — and that distinction exists to remind you what you can destroy.

# The Principle of Least Privilege

**Give each user only the permissions they need. Nothing more.**

```
                root (superuser)
                     |
    +----------------+----------------+
    |                |                |
sysadmin         developer         webapp
(full sudo)    (deploy only)     (no sudo)
```

**Why this matters:**

### Limits Blast Radius of Mistakes

`rm -rf /` as root? Fatal.

### Limits Blast Radius of Breaches

Compromised app ≠ compromised system

### Creates Audit Trails

Know WHO did what

### Matches Cloud IAM Philosophy

AWS/GCP use the same principle

> Your new developers don't need root to write code. Don't give it to them.

# Where User Info Lives: /etc/passwd and /etc/shadow

Every user has a line in `/etc/passwd`:

```
anas:x:1000:1000:Anas:/home/anas:/bin/bash
```

| Position | Value | Meaning |
|---|---|---|
| 1 | `anas` | Username |
| 2 | `x` | Password stored elsewhere |
| 3 | `1000` | User ID (UID) |
| 4 | `1000` | Primary Group ID (GID) |

`/etc/passwd` is world-readable — anyone can see it. That's why the actual password hash is in `/etc/shadow`, which only root can read. The `x` in the password field is a pointer to shadow.

# Demo: /etc/passwd

```
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
anas:x:1000:1000:Anas:/home/anas:/bin/bash
```

| Key detail | Meaning |
| --- | --- |
| `x` in password field | Actual hashes stored in `/etc/shadow` instead |

# Demo: /etc/shadow

```
sudo cat /etc/shadow
root:*:19500:0:99999:7:::
anas:$6$rounds=5000$salt$hash...:19500:0:99999:7:::
khalid:!:19500:0:99999:7:::
```

| Key detail | Meaning |
| --- | --- |
| `sudo` required | Only root can read shadow — that separation is the security model |

# Where Group Info Lives: /etc/group

Groups let you give shared permissions to a set of users.

| Field | Meaning | Example |
|---|---|---|
| Group name | The label | `sudo` |
| Group ID | Numeric ID | `27` |
| Member list | Comma-separated users | `anas,khalid` |

**Common system groups:**

| Group | Purpose |
|---|---|
| `sudo` | Can use sudo |
| `www-data` | Web server processes |
| `docker` | Can run Docker commands |
| `adm` | Can read system logs |

# Demo: /etc/group

```
cat /etc/group
root:x:0:
sudo:x:27:anas
docker:x:998:anas
developers:x:1002:anas,khalid
```

| Key detail | Meaning |
| --- | --- |
| `sudo:x:27:anas` | `anas` is in the sudo group — can elevate privileges |

# Who Am I? — Identity Commands

These are the first commands you'll run on any new server:

| Command | What It Shows |
|---|---|
| `whoami` | Your current username |
| `id` | Full identity — UID, GID, all groups |
| `id username` | Another user's identity without switching |
| `groups` | Your group memberships (names only) |

> **Pro tip:** `id -Gn username` shows all groups a user belongs to — fastest way to check permissions.

# Demo: Identity Commands

```
whoami
anas

id
uid=1000(anas) gid=1000(anas) groups=1000(anas),27(sudo),998(docker)

sudo whoami
root
```

| Key detail | Meaning |
| --- | --- |
| `sudo whoami` returns `root` | sudo temporarily switches identity to UID 0 |

# Who Else Is Here? — Activity Commands

On a shared server, you need to know who else is logged in:

| Command | What It Shows |
| --- | --- |
| `w` | Who's logged in, from where, doing what |
| `last -5` | Last 5 login sessions with timestamps |

# Demo: Activity Commands

```
w
USER   TTY   FROM           LOGIN@  WHAT
anas   pts/0 192.168.1.10   09:00   -bash
khalid pts/1 10.0.0.5       09:15   vim app.py

last -5
anas  pts/0  192.168.1.10  Mon Mar 2 09:00   still logged in
root  pts/1  192.168.1.1   Sun Mar 1 22:00 - 22:05
```

| Key detail | Meaning |
| --- | --- |
| `w` shows FROM column | See which IP each user connected from |

# Creating Users

`adduser` — interactive, creates home, prompts for password

```
sudo adduser khalid
```

`useradd` — low-level, use in scripts

```
sudo useradd -m -s /bin/bash -G developers khalid
sudo passwd khalid     # set password as separate step
```

| Feature | `adduser` | `useradd` |
|---|---|---|
| Home directory | Automatic | Must use `-m` |
| Password | Prompts immediately | Separate `passwd` step |
| Interactive | Yes | No |
| Best for | Manual setup | Scripts/automation |

# Demo: Creating a User

```
sudo useradd -m -s /bin/bash -G developers khalid
sudo passwd khalid
New password: ••••••••
Retype new password: ••••••••
passwd: password updated successfully

id khalid
uid=1001(khalid) gid=1001(khalid) groups=1001(khalid),1002(developers)
```

| Key detail | Meaning |
| --- | --- |
| `passwd` as separate step | `useradd` doesn't set a password — you do it after |

# Modifying and Deleting Users

`usermod` — change user properties

```
sudo usermod -aG developers khalid    # SAFE: append to group
sudo usermod -G developers khalid     # DANGER: replaces all groups!
sudo usermod -L khalid                # lock account
sudo usermod -U khalid                # unlock account
```

`userdel` — delete permanently

```
sudo userdel khalid          # keep home directory
sudo userdel -r khalid       # delete user AND home directory
```

> Always use `-aG` (append + group). `-G` alone **replaces** all other group memberships — it has ended careers.

# Demo: Modifying a User

```
sudo usermod -aG developers khalid   # safe: append
id khalid
uid=1001(khalid) gid=1001(khalid) groups=1001(khalid),1002(developers)

sudo usermod -L khalid               # lock account
sudo usermod -U khalid               # unlock account
```

| Key detail | Meaning |
| --- | --- |
| `-aG` (append + group) | The `a` is critical — without it, all other groups are removed |

# Managing Groups and Shared Directories

**Create group, add members, share a directory:**

```
# Create the group
sudo groupadd developers

# Add members
sudo gpasswd -a anas developers
sudo gpasswd -a khalid developers

# Set group ownership + permissions on shared directory
sudo chown -R :developers /var/www/project
sudo chmod -R g+w /var/www/project
sudo chmod g+s /var/www/project      # setgid: new files inherit the group
```

`g+s` **(setgid bit)** ensures files created inside automatically inherit the group — Anas's new files are immediately accessible to Khalid.

> List members: `getent group developers`

# Demo: Shared Directory Setup

```
sudo groupadd developers
sudo gpasswd -a anas developers
sudo gpasswd -a khalid developers
sudo chown -R :developers /var/www/project
sudo chmod -R g+w /var/www/project
sudo chmod g+s /var/www/project

ls -la /var/www/
drwxrwsr-x 2 root developers 4096 Mar 2 project
```

| Key detail | Meaning |
| --- | --- |
| `chmod g+s` (setgid) | New files auto-inherit the group — no manual chown needed |

# What Can Your Users Actually Do?

Your developers have accounts. They can log in. But now what?

## Regular users can:

- Read their own files
- Write in their home directory
- Run basic commands

## Regular users cannot:

- Install software
- Manage services
- Edit system configuration
- Access other users' files

> **How do you give them elevated access without giving them root? Enter `sudo`.**

# Root, su, and sudo

**Root (UID 0)** is the superuser — it can do anything. That's the problem.

| Problem with root access | Consequence |
|---|---|
| No audit trail | Can't tell WHO did what |
| Typos are fatal | `rm -rf /` has no guardrails |
| No fine-grained control | All or nothing |
| Security breach | Attacker gets everything |

`su` switches you to another user (requires their password).
`sudo` runs ONE command with elevated privileges (uses YOUR password).

```
sudo apt update          # run one command as root
sudo -u deploy whoami    # run as a specific user
sudo -l                  # list your sudo privileges
```

> Use `sudo` for daily admin work. `su` is rare — mostly debugging.

# /etc/sudoers — Configuring Access

**NEVER edit** `/etc/sudoers` **directly.** Use `visudo` — it validates syntax before saving. A syntax error can lock you out of sudo entirely.

```
sudo visudo
```

**The sudoers syntax:**

```
deploy ALL=(ALL:ALL) ALL
  |     |     |  |    |
  |     |     |  |    +-- Commands allowed
  |     |     |  +------- As any group
  |     |     +---------- As any user
  |     +---------------- On any host
  +---------------------- Username (or %groupname for a group)
```

```
# Full admin access
deploy ALL=(ALL:ALL) ALL

# Specific command, no password prompt
deploy ALL=(ALL) NOPASSWD: /usr/bin/cp /tmp/app/* /var/www/app/

# Group-based (%groupname)
%developers ALL=(ALL) /usr/bin/apt install
```

> Note: once you learn `systemctl` in M11, you'll add service-restart rules here. The syntax is the same.

# sudoers.d — One File Per Role

**Cleaner approach:** Create one file per role in `/etc/sudoers.d/` instead of editing the main sudoers file.

```
sudo visudo -f /etc/sudoers.d/deploy
```

Example file content:

```
# /etc/sudoers.d/deploy
deploy ALL=(ALL) NOPASSWD: /usr/bin/cp /tmp/app/* /var/www/app/
deploy ALL=(ALL) NOPASSWD: /usr/bin/cat /var/log/app.log
```

**Benefits:**

### Organized by role

Easy to add or remove

### No risk to main sudoers file

Changes are isolated

### Auto-included

Files in `/etc/sudoers.d/` are included automatically

# Aha! Moment 💡

| Linux Concept | Cloud Equivalent | Same Principle |
|---|---|---|
| Users (UID) | IAM Users | Individual identities |
| Groups | IAM Groups | Organize by role |
| sudo rules | IAM Policies | Grant specific permissions |
| Least privilege | IAM Best Practices | Minimum necessary access |

With sudo, your CI/CD pipeline can deploy without root passwords:

```
# CI/CD user deploys — no root password needed
deploy ALL=(ALL) NOPASSWD: /usr/bin/cp /tmp/build/* /var/www/app/

# Each person uses their own password — audit logs show WHO
# "anas used sudo at 14:30" — "khalid used sudo at 15:45"
```

> You're not just learning Linux — you're learning the identity management pattern that every cloud platform reuses.

# Quick Reference

**User management:**

| Task | Command |
|---|---|
| Check identity | `whoami`, `id`, `groups` |
| See who's online | `w`, `last -5` |
| Create user (interactive) | `sudo adduser username` |
| Create user (script) | `sudo useradd -m -s /bin/bash username` |
| Add to group (safe) | `sudo usermod -aG group user` |
| Lock / Unlock | `sudo usermod -L user` / `sudo usermod -U user` |
| Delete + home | `sudo userdel -r user` |

**Group management:**

| Task | Command |
|---|---|
| Create group | `sudo groupadd name` |
| Add member | `sudo gpasswd -a user group` |
| Remove member | `sudo gpasswd -d user group` |
| List members | `getent group name` |

**sudo:**

| Task | Command |
|---|---|
| Run command as root | `sudo command` |
| List your privileges | `sudo -l` |
| Edit sudoers safely | `sudo visudo` |
| Create role file | `sudo visudo -f /etc/sudoers.d/rolename` |

**Key files:** `/etc/passwd` · `/etc/shadow` · `/etc/group` · `/etc/sudoers`

# Mission Complete: Users, Groups & Sudo ✅

**You set up a team from scratch — accounts, groups, shared access, and precision permissions.**

- → `$` **= regular user (UID 1000+),** `#` **= root (UID 0)** — the prompt reflects who you are
- → `/etc/passwd` — accounts; `/etc/shadow` — passwords; `/etc/group` — memberships
- → `adduser` for interactive setup; `useradd` for scripts
- → `usermod -aG` to add to groups — never `-G` alone
- → `sudo` runs one command with elevation; always via `visudo` for configuration
- → `/etc/sudoers.d/` — one file per role keeps things clean and reversible

> Next mission: your team has accounts and permissions. Now they need to connect securely from the outside world — and 10,000 bots are hammering your server right now. SSH keys make passwords obsolete.

# Questions?